

Static Chat → LLM Chat

A Conversion Guidebook — Turning scripted, button-driven chat flows into an LLM-powered conversation engine using Google Gemma

Prepared 23 June 2026 • Reference implementation: Conversational UI (Generative BI Report Hub) • Audience: engineers + product / non-technical stakeholders

What this is. Our team took a prototype whose chat was *static* — every reply was hardcoded, every path scripted in advance — and rebuilt it as an *LLM-based* system where Google Gemma understands free-text questions, picks the right data, and assembles the answer live. This guidebook is the playbook for doing the same to **any** static-chat prototype in the company. It explains the idea in plain language, lays out the exact architecture and tools, gives copy-ready code patterns from our real implementation, and ends with a step-by-step migration checklist.

Read it two ways. Sections marked **PLAIN** are written for anyone. Sections marked **BUILD** are for the engineer doing the wiring. Every reader should at least read Parts 1–3.

Contents

- | | |
|--|--|
| 1. Static vs. LLM — the core idea | 11. Your data layer |
| 2. What "good" looks like (our result) | 12. No database? A Markdown context file |
| 3. The mental model & architecture | 13. Clarification & follow-up flows |
| 4. Tools, accounts & prerequisites | 14. Caching, retries & safety nets |
| 5. Meet Gemma — the model we use | 15. Security & configuration |
| 6. The backend pipeline, step by step | 16. Testing & evaluation |
| 7. Talking to Gemma (the API) | 17. Cost & performance |
| 8. Prompt engineering — the real work | 18. Migration checklist |
| 9. Streaming answers (SSE) | 19. Pitfalls & FAQ |
| 10. The frontend conversion | 20. Glossary (plain English) |

1. Static vs. LLM — the core idea PLAIN

A **static chat flow** is a decision tree someone drew in advance. The user clicks "Sales", then "Monthly Revenue", then "Last 12 months", and the app shows a pre-built screen. Every word the bot says was typed by a developer. It cannot answer anything that wasn't anticipated. If the user types "why did the West region dip in March?", a static bot has no branch for that — it shrugs.

An **LLM-based chat flow** replaces the decision tree with a *language model* — software that reads the user's actual words, figures out what they mean, and decides what to do next. Instead of "if button == X, show screen Y", it is "read the question, understand intent, fetch the right data, and compose a fresh answer."

Analogy. A static chat is a *vending machine*: fixed buttons, fixed slots, exact-change only. An LLM chat is a *knowledgeable shop assistant*: you describe what you want in your own words, they ask a clarifying question if needed, then they go find it. Our job in this conversion is to keep the same storefront (the UI) but swap the vending machine behind it for the assistant.

What actually changes

Aspect	Static chat (before)	LLM chat (after)
Understanding input	Button clicks / keyword match	Gemma reads free-text and infers intent
Deciding the response	<code>if/else</code> rules hardcoded	Model chooses an action (route, clarify, answer, edit)
The answer content	Pre-written templates	Generated live from real data
Handling the unexpected	"I didn't understand that"	Asks a smart clarifying question, or answers anyway
Adding a new topic	Code a new branch + screens	Add the data source; the model adapts
Follow-ups ("now by region")	Usually impossible	Model edits the existing answer in place

The one-sentence version. We stop writing the conversation and start writing the *rules of the conversation*, then let Gemma run it within those rules — always grounded in **your real data**, never free-floating.

A critical principle: "Deterministic-First"

The biggest mistake teams make is letting the LLM do *everything*. That is slow, expensive, and unpredictable. Our design uses the LLM only for the **judgement calls a human would make** — "what does this person want?", "which chart tells this story best?" — and uses plain, predictable code for everything else (fetching rows, formatting numbers, validating output). We call this **Deterministic-First**: the pipeline is the skeleton, the LLM is the brain. This keeps the system fast, cheap, and debuggable.

2. What "good" looks like — our result PLAIN

For context, here is what our converted system can now do that the static version could not:

- Accept a typed question like *"compare territory T-007 and T-001 return rate"* and produce a tailored comparison with the real numbers.
- Ask a clarifying question — *"Which domain: Sales, Network, Contact Center, or Customer Experience?"* — only when genuinely needed, never robotically.
- Stream the answer in as it builds (status → title → each chart appears one by one), so it feels alive instead of a long blank wait.
- Accept a follow-up — *"now show only the KPIs"* or *"summarize this in bullet points"* — and edit the answer it already produced, without re-running everything.

- Pick the *right* visualization for the data automatically: a trend over time becomes a line chart, a ranking becomes a ranked list, a single number becomes a KPI card.

1

LLM call to classify *and* edit a follow-up (fused)

20+

UI component types the model can choose from

5 min

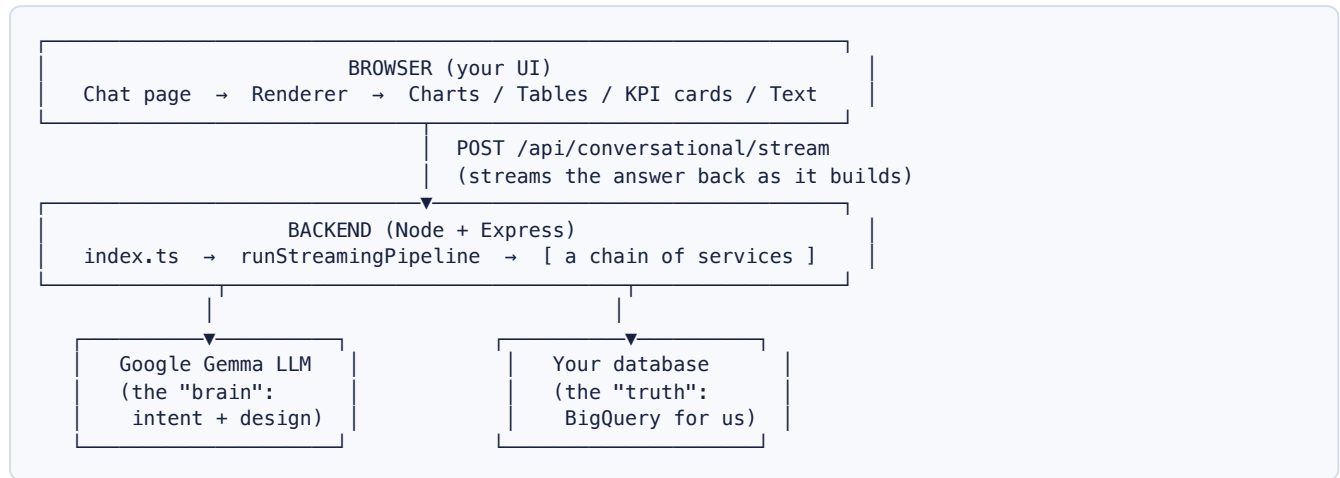
cache TTL — repeat questions are instant

x3

auto-retries on a flaky model call

3. The mental model & architecture BUILD

Picture four boxes. The browser shows the chat. A small backend server is the traffic controller. Gemma is the brain it consults. Your database is where the truth lives. The backend never lets the model touch the database directly — it fetches data itself, hands the model a small sample, and asks only for *decisions*.



The three jobs the model does for us

Job	Plain meaning	When it runs
Analyze / route	"What is this person asking, and which data answers it? Or do I need to ask them something first?"	Every new message
Design report	"Given this data, what's the best way to show the answer — which charts, which numbers, what title?"	After data is fetched
Classify & edit	"They have an answer on screen and said something new — do they want an edit, a fresh answer, or just a written reply? Apply it."	On a follow-up to an existing answer

Why a backend at all? Three reasons, all non-negotiable. (1) Your API keys must never ship to the browser — they live only on the server. (2) The browser can't safely hold database credentials. (3) The "fetch data → sample it → ask the model → validate → stream" pipeline is server work. The browser only renders.

4. Tools, accounts & prerequisites BUILD

Here is everything a team needs before writing a line of conversion code. Items are tagged MUST (required), SHOULD (strongly recommended), NICE (optional polish).

Tool / account	What it's for	Tag
Google AI API key (<code>GOOGLE_AI_API_KEY</code>)	Access to Gemma. Get it from Google AI Studio. This is the single most important credential.	MUST
@google/genai npm package	The official Google SDK to call Gemma from Node.	MUST
Node.js 18+	Runtime for the backend server.	MUST
Express.js	The web-server framework that exposes the chat endpoint.	MUST
TypeScript	Type-safe code — catches whole classes of bugs when wiring model output to UI.	SHOULD

A data source	Where your real answers come from. We use Google BigQuery; yours could be Postgres, an internal API, a CSV, anything.	MUST
A frontend (React in our case)	Your existing UI. You keep it; you only rewire how it talks to the backend.	MUST
A component renderer	One function that turns <code>{ type, props, data }</code> from the model into actual UI widgets.	MUST
Charting library (Recharts/amCharts)	Draws the charts the model asks for.	SHOULD
An in-memory cache	Avoids paying for the same question twice. Can be a simple Map with a timestamp.	SHOULD
Evaluation test set	A list of example questions + expected behaviour, to measure quality as you tune prompts.	SHOULD
BigQuery service account (if using BQ off-GCP)	Roles: <i>BigQuery Data Viewer</i> + <i>Job User</i> . Only if your data lives in BigQuery.	NICE

INSTALL, IN ONE BLOCK

```
# backend
mkdir backend && cd backend && npm init -y
npm install express dotenv @google/genai
npm install -D typescript ts-node @types/express @types/node
# if your data is in BigQuery:
npm install @google-cloud/bigquery
```

THE `.ENV` FILE (BACKEND ONLY — NEVER COMMIT, NEVER SHIP TO BROWSER)

```
PORT=3001
GOOGLE_AI_API_KEY=your_key_here
# data source creds — example for BigQuery:
BQ_PROJECT_ID=your-project
BQ_DATASET=your_dataset
BQ_CLIENT_EMAIL=service-account@your-project.iam.gserviceaccount.com
BQ_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\n...\n-----END PRIVATE KEY-----\n"
# which frontend origin may call this backend (CORS):
FRONTEND_ORIGIN=http://localhost:5173
```

5. Meet Gemma — the model we use PLAIN

Gemma is Google's family of open language models, served through the same Google AI API as Gemini. In our system the exact model id is `gemma-4-31b-it` (the instruction-tuned, ~31-billion-parameter Gemma). You call it over the network; you don't host it.

Why Gemma for this job?

- **Good enough, cheaper.** Intent classification and chart selection don't need a frontier model. Gemma handles them at lower cost and latency.
- **JSON-reliable.** With "JSON mode" on, Gemma returns structured data our code can parse directly — essential, because we feed its output straight into the UI.
- **One SDK.** The same `@google/genai` client and the same API key reach both Gemma and Gemini, so swapping models later is a one-line change.

Model choice is a dial, not a commitment. The model id lives in exactly one constant (`const MODEL = 'gemma-4-31b-it'`). If a task needs more reasoning power, point that constant at a larger model; if you need cheaper, point it lower. Nothing else changes. Start with Gemma; upgrade only the calls that demonstrably need it.

THREE THINGS TO KNOW ABOUT HOW GEMMA BEHAVES

- **It can ramble or "think out loud."** Some responses include `<think>...</think>` blocks or prose around the JSON. We strip those before parsing (see `stripThinkTags` / `extractJSON` in Part 7).
- **It is non-deterministic.** The same prompt can give slightly different wording. We pin `temperature` low (0.1–0.2) for decisions, a touch higher (0.4) for creative report design.
- **It occasionally fails.** Network blips, rate limits (429), server errors (500). We wrap every call in a retry-with-backoff and always have a deterministic fallback.

6. The backend pipeline, step by step BUILD

This is the heart of the conversion. When a message arrives, it flows through an ordered pipeline. Each step is small, testable, and mostly plain code — the model is consulted at only three points. Below is the exact flow from our `runStreamingPipeline`.

```
USER MESSAGE arrives at POST /api/conversational/stream
|
[0] CACHE CHECK ..... same question recently? → replay saved answer, done.
|                       (key = hash of query + persona + history + context)
|
[1] ANALYZE / ROUTE .... Gemma: enough info to answer, or must I ask first?
|                       └─ "clarify" → send a question to the UI → STOP
|                       └─ "route"  → pick the exact data table → continue
|
[2] FOLLOW-UP HANDLER .. only if an answer is already on screen:
|                       Gemma (1 fused call): edit layout? re-query? just
|                       reply in text? start fresh? → may STOP here
|
[3] FETCH DATA ..... run the query against your database. 0 rows? →
|                       recover by suggesting alternative tables.
|
[4] ANALYZE SHAPE ..... plain code: which columns are numbers, categories,
|                       dates? how many distinct values? is it a time series?
|
[5] PRE-AGGREGATE ..... plain code: dedupe & summarise rows, push the rows
|                       the user actually asked about to the front.
|
[6] DESIGN REPORT ..... Gemma: given this data shape + 10–20 sample rows,
```

```

| choose title, charts, KPIs, follow-up suggestions.
↓
[7] FIX COLUMN CASING .. plain code: the model may lowercase a column the DB
| spells UPPERCASE – map it back to the real name.
↓
[8] HYDRATE ..... plain code: attach the REAL data arrays to each chart
| the model designed. (The model never sees all rows.)
↓
[9] STREAM & CACHE ..... send title, then each component, then suggestions,
| then "done". Save the result in cache.

```

Read the table of consult points. The model is called at **[1]**, **[2]**, and **[6]** only. Steps 0, 3, 4, 5, 7, 8, 9 are ordinary, predictable code. That ratio — three smart steps, six dumb ones — is what keeps the system fast and affordable.

Why each "dumb" step matters

Step	What goes wrong without it
[0] Cache	You pay the model (and the wait) for identical repeat questions.
[4] Shape analysis	The model guesses column types and picks a pie chart for a date column.
[5] Pre-aggregate	The model sees 5,000 raw rows, blows the token budget, and averages the wrong thing.
[7] Casing fix	The chart renders blank because <code>revenue</code> $\neq$ the DB's <code>REVENUE</code> .
[8] Hydration	Charts show the 10 sample rows instead of the full dataset.

7. Talking to Gemma — the API BUILD

One client, created once and reused. Every call uses the same shape: a system instruction (the rules), the user content (the question + data), and a config block (model, temperature, JSON mode).

SETUP

```
import { GoogleGenAI, Type } from '@google/genai';

const MODEL = 'gemma-4-31b-it';

let _ai = null;
function getAI() {
  if (!_ai) _ai = new GoogleGenAI({ apiKey: process.env.GOOGLE_AI_API_KEY });
  return _ai;
}
```

A SINGLE CALL (JSON MODE — THE PATTERN WE USE EVERYWHERE)

```
const response = await ai.models.generateContent({
  model: MODEL,
  config: {
    temperature: 0.2, // low = consistent decisions
    maxOutputTokens: 768,
    responseMimeType: 'application/json', // ← forces structured output
    systemInstruction: system, // the rules (see Part 8)
  },
  contents: [{ role: 'user', parts: [{ text: user }] }],
});

const raw = response.text ?? '';
const parsed = JSON.parse(extractJSON(stripThinkTags(raw)));
```

THE TWO CLEANERS YOU WILL ALWAYS NEED

```
// remove the model's optional "thinking out loud" blocks
function stripThinkTags(t) {
  return t.replace(/<think>[\s\S]*?<\/think>/gi, '').trim();
}

// grab just the { ... } even if prose leaked around it
function extractJSON(t) {
  const s = t.indexOf('{'), e = t.lastIndexOf('}');
  return (s !== -1 && e > s) ? t.slice(s, e + 1) : t;
}
```

RETRY WITH BACKOFF — WRAP EVERY MODEL CALL

```
async function withRetry(fn, maxAttempts = 3) {
  let lastErr;
  for (let a = 1; a <= maxAttempts; a++) {
    try { return await fn(); }
    catch (err) {
      lastErr = err;
      const code = err?.error?.code ?? err?.status ?? 0;
      if (code !== 500 && code !== 429) throw err; // only retry transient
      if (a < maxAttempts)
        await new Promise(r => setTimeout(r, Math.pow(2, a) * 600)); // 1.2s, 2.4s
    }
  }
}
```



```
throw lastErr;
}
```

JSON mode vs. function calling. The Google SDK also supports "function calling" (the model picks a named tool). We tried it and found plain **JSON mode** (`responseMimeType: 'application/json'`) more reliable with Gemma. We define our tool schemas anyway for documentation, but the calls that matter use JSON mode and we parse the result ourselves. If you are on Gemini instead, function calling works well too — pick one and be consistent.

8. Prompt engineering — the real work BUILD

This is where 80% of the quality lives. A prompt is the instruction sheet you hand the model. In a static chat, you wrote the answers; here, you write the *rules for producing answers*. Three prompts carry our system.

8.1 The analyze/route prompt — decide before doing

Goal: read the question, decide "route" (I know enough) or "clarify" (I must ask). The decisive trick is **giving the model a closed list of real options** and forbidding it to invent any.

```
You are a friendly business intelligence assistant...

AVAILABLE DATA (only suggest options from this list – these are the
only reports with real data):
– domain="Sales"    report="Monthly Revenue"  table="fact_sug_monthly_rollup"
– domain="Network"  report="Network KPI Trends" table="fact_network_kpi_points"
...(generated from your real data sources)...

RULES:
1. If query + history name a domain AND a report → action="route",
   set table to the EXACT string above.
2. If domain unclear → action="clarify", options = the domain names above.
3. If domain clear but report unclear → action="clarify",
   options = ONLY that domain's report names.
5. NEVER invent table, domain, or report names.

Respond with valid JSON only:
{ "action":"route"|"clarify", "opener":"...", "question":"...",
  "options":["..."], "table":"...", "intent":"trend"|"comparison"|"metric_by_dimension" }
```

The golden rule of grounding. Notice rules 5 and 6 forbid invention, and the options are validated *again in code* after the model replies — anything not in your real catalog is dropped. An LLM will happily offer a "2023 Cohort Analysis" report that doesn't exist. Constrain it in the prompt **and** verify in code. Never trust, always check.

8.2 The design-report prompt — choose the visualization

Goal: given the data's shape and a few sample rows, design the answer. This prompt is long because it teaches the model *which component fits which intent*. Excerpt of the decision rules we give it:

```
STEP 1 – UNDERSTAND THE QUERY INTENT:
TREND / over time      → LineChart or AreaChart + 1 KPICard
COMPARISON / ranking   → BarChart or RankedList or ComparisonCard
SINGLE METRIC           → KPIGrid or 1–3 KPICards, no table unless asked
CORRELATION            → ScatterPlot + 1 InsightCard
SUMMARY / "explain/why" → 1–2 KPICards + InsightCard, NO charts

COLUMN NAME RULE (most important):
Copy column names from EXACT_COLUMNS character-for-character into
xKey, yKey, nameKey, valueKey. NEVER lowercase or invent them.
```

ENTITY SPECIFICITY:

If the query names T-007, use THAT row's values for KPIs — never the global average. Title and message must name the entity explicitly.

Respond with JSON only:

```
{ "title": "...", "message": "...", "cards": [{ "renderType": "...", "props": {...} }],  
  "followUp": [{ "label": "...", "intent": "..." } ] }
```

Why we send only a sample, never all rows. Step 5 of the pipeline hands the model ~10–20 representative rows plus a written summary of the shape ("12 columns, 3 are dates, REVENUE ranges 0–4.2M"). The model designs the layout from the sample; step 8 attaches the *full* data afterward. This keeps prompts small (cheap, fast) and stops the model from miscalculating totals.

8.3 The fused follow-up prompt — one call, four outcomes

When an answer is already on screen and the user types again, a naïve design makes two model calls (classify, then act). We fused them into **one** call that both decides and does:

The user has an open report and sends a follow-up. Choose ONE action:

```
QA / ANSWER ("summarize", "why", "explain", "in bullets", any question)  
  → { "action": "qa_answer", "message": "...uses real values...", "followUp": [...] }  
STRUCTURAL EDIT ("remove the chart", "show only KPIs", "change to line")  
  → { "action": "edit_structural", "acknowledgment": "...", "cards": [surviving cards] }  
DATA CHANGE ("top 5", "filter T-017", "last 3 months")  
  → { "action": "edit_data_change", "sqlOverride": "ORDER BY REVENUE DESC LIMIT 5" }  
NEW REPORT (unrelated topic) → { "action": "new_report" }  
AMBIGUOUS → { "action": "clarify_intent" }
```

Three habits that make prompts reliable. (1) Always end with the exact JSON shape you expect. (2) Give a closed list of allowed values (render types, table names, actions) — never an open question. (3) Put the most important rule ("never invent column names") in CAPS near the top *and* repeat it at the bottom. Models weight the start and end of a prompt most.

9. Streaming answers (SSE) BUILD

A full report can take a few seconds. Rather than a long blank wait, we **stream** the answer: the status appears, then the title, then each chart pops in as it's ready. The mechanism is **Server-Sent Events (SSE)** — a one-way stream of small messages from server to browser over a single HTTP response.

Analogy. A normal API call is a sealed letter — nothing until it all arrives. SSE is a phone call — you hear each sentence as it's spoken. The user sees progress, so the wait feels short.

SERVER SIDE — A TINY **SEND** HELPER

```
res.writeHead(200, {
  'Content-Type': 'text/event-stream',
  'Cache-Control': 'no-cache',
  'Connection': 'keep-alive',
});
const send = (event, data) =>
  res.write(`event: ${event}\ndata: ${JSON.stringify(data)}\n\n`);

send('status',    { message: 'Understanding your query...' });
send('meta',      { title, description, template });
send('component', hydratedCard); // one per chart/KPI
send('followUp',  suggestions);
send('done',      { success: true });
```

THE EVENT PROTOCOL (WHAT THE BROWSER LISTENS FOR)

Event	Meaning for the UI
status	Show a progress line ("Querying database...")
meta	Set the answer's title + description
component	Append one chart/table/KPI — render it immediately
followUp	Show suggested next-question chips
clarification	Show a question card with option buttons
qa_answer	Show a text reply, no new dashboard
acknowledgment	Confirm an edit was applied
error	Show an error state + retry button
done	Stream finished — stop the spinner

BROWSER SIDE — READ THE STREAM

```
const res = await fetch('/api/conversational/stream', { method: 'POST', body });
const reader = res.body.getReader();
// loop: decode each chunk, split on \n, route by event type, patch UI state
```

Don't have time for SSE on day one? Ship a non-streaming endpoint first (one JSON response with the whole answer) and add streaming later. Our backend keeps both: `/api/conversational/stream` (SSE) and `/api/conversational` (plain JSON fallback). Same pipeline underneath.

10. The frontend conversion

BUILD

Good news: you keep your UI. The chat bubbles, the input box, the sidebar — all stay. What changes is *where the answer comes from*. In the static version, a click looked up a hardcoded screen. Now, the message goes to the backend and the answer streams back as a list of components to render.

The one component that makes this work: a renderer

The model returns abstract descriptions like `{ renderType: "LineChart", props: { xKey: "MONTH", yKey: "REVENUE" }, data: [...] }`. You need **one** function — call it `UITreeRenderer` — that maps each `renderType` to a real widget. This is the bridge between "what the model decided" and "what the user sees."

```
function UITreeRenderer({ renderType, props, data }) {
  switch (renderType) {
    case 'LineChart': return <LineChart {...props} data={data} />;
    case 'BarChart':  return <BarChart {...props} data={data} />;
    case 'KPICard':   return <KPICard {...props} />;
    case 'RankedList': return <RankedList {...props} data={data} />;
    case 'Table':     return <DataTable {...props} data={data} />;
    // ...20+ types: charts, KPIs, narrative cards, layout wrappers
    default:          return <Fallback type={renderType} />;
  }
}
```

Build the renderer's vocabulary first. The set of `renderType` values your renderer understands *is* the set the model is allowed to choose from. List them in the design prompt (Part 8.2) and implement each in the renderer. Start with five (KPICard, LineChart, BarChart, Table, InsightCard) and grow. A model asking for a type you didn't build should hit the `default` fallback, never crash.

What to delete from the static frontend

Static artifact	Fate
Hardcoded decision-tree branches (<code>if step==='X'</code>)	Delete — the model decides now
Pre-written bot reply strings	Delete — generated live
Fixed button menus for every step	Replace with model-supplied <code>followUp</code> chips + <code>clarification</code> options
Chat bubbles, input box, sidebar, layout	Keep — purely visual
Your chart/table components	Keep — the renderer now drives them

Message state shape

Each message becomes richer than a string. Store the rendered components alongside the text so history reloads correctly:

```
Message = {
  role: 'user' | 'assistant',
  content: string,           // the text
  components: UINode[],      // hydrated charts/tables/KPIs
  suggestedPrompts: [{ label, intent }],
}
```

11. Your data layer BUILD

The model is only as good as the data you ground it in. Your job is to give it a clean, named menu of data sources. We use a **data source map** — a plain list that ties a human report name to a real table and its key metrics.

```
DATA_SOURCES = [  
  { domain: 'Sales',   reportName: 'Monthly Revenue & Take Rate',  
    table: 'fact_sug_monthly_rollup',  
    kpis: ['Revenue', 'Run Rate', 'Take Rate %', 'Return Rate %'] },  
  { domain: 'Network', reportName: 'Network KPI Trends',  
    table: 'fact_network_kpi_points',  
    kpis: ['Network KPI Score', 'Signal Strength', 'Latency'] },  
  // ...one entry per real report...  
];
```

This single list powers three things: the options the model offers when clarifying, the `enum` of valid tables it may route to, and the catalog text injected into the analyze prompt. **If it isn't in this list, the model can't offer it** — which is exactly the safety you want.

TWO DATA-LAYER HABITS WORTH COPYING

- **Startup probe.** On boot, run `SELECT 1 FROM table LIMIT 1` against each source and cache which ones actually return rows. Only offer reports backed by live data. (We re-probe every 24h.)
- **Catalog refresh.** Periodically pull real column names + descriptions from your warehouse into a markdown "catalog" and inject it into prompts. This lets the model ask field-aware clarifying questions ("by territory or by device?") instead of generic ones.

Not using BigQuery? Nothing above is BigQuery-specific. Swap the `table` field for an API endpoint, a SQL view, a collection name, or a file path. The pipeline only needs a function that takes a source id (+ optional filter) and returns `rows[]`. Everything downstream — shape analysis, hydration, rendering — is data-source agnostic.

11½. No database? Use a Markdown context file BUILD

Many prototypes have **no warehouse at all** — no BigQuery, no SQL, just knowledge that lives in someone's head or a wiki. You don't need a database to go LLM-based. Instead, write the knowledge down as a plain **Markdown (.md)** file in the backend, and inject it into the model's system instruction on every call. The model reads it like an open-book exam and answers from it.

Analogy. BigQuery is a *filing cabinet* the assistant searches. A Markdown context file is a *cheat-sheet* you tape to the assistant's desk — everything it needs to know, always in view. For small, mostly-static knowledge, the cheat-sheet is simpler, faster, and free.

We already do exactly this. Even in our BigQuery build, the model is handed a hand-shaped Markdown catalog (`backend/data/catalog_context.md`) describing domains, reports, and column names. It's loaded once, cached for 25 hours, and injected into the analyze prompt. The no-database path is the *same mechanism* — you just author the file by hand instead of generating it from a warehouse.

When the Markdown file is the right choice

Use a Markdown context file when...	Use a real database when...
Knowledge is small (fits in a few pages)	Data is large (thousands+ of rows)
Facts change rarely (policies, FAQs, product specs)	Numbers change constantly (live metrics)
Answers are text/explanations	Answers are charts/tables of figures
No data team or warehouse available	You already have SQL access
You want to ship this week	You need always-current aggregates

What goes in the file

Everything the model would otherwise have to guess. Structure it with clear headings so the model can navigate it. Typical contents:

- **Who the assistant is** and its scope ("You help employees with HR policy questions only").
- **The knowledge itself** — policies, FAQs, product details, process steps, definitions.
- **The closed list of topics** it can route to (the same grounding idea as `DATA_SOURCES`).
- **Worked examples** — a few question→answer pairs that show the tone and format you want.
- **What to do when it doesn't know** ("If the answer isn't in this document, say so and offer to escalate").

EXAMPLE — `BACKEND/DATA/KNOWLEDGE.MD`

```
# Acme Support Assistant – Knowledge Base
Last updated: 2026-06-20

## Scope
You answer questions about Acme's return policy, shipping, and warranties.
If a question falls outside these topics, say you can't help and suggest
contacting support@acme.com.

## Topics (only route to these)
- Returns
- Shipping
- Warranty

## Returns
```

- Window: 30 days from delivery for a full refund.
- Condition: unused, original packaging.
- Final-sale items (clearance) cannot be returned.

Shipping

- Standard: 5-7 business days, free over \$50.
- Express: 2 business days, \$12 flat.

Warranty

- Electronics: 1 year against manufacturing defects.
- Claims need the order number and a photo.

Examples

Q: "Can I return a clearance shirt?"

A: "Clearance items are final sale and can't be returned. Anything else I can help with?"

How to load and inject it (grounded in our real loader)

Read the file once, cache it in memory, and prepend it to the system instruction. This is the same shape as our `loadCatalogContext()`, minus the database refresh.

```
import fs from 'fs/promises';
import path from 'path';

const KB_PATH = path.resolve(__dirname, '../data/knowledge.md');
let _kb = null, _at = 0;
const TTL = 25 * 60 * 60 * 1000; // re-read at most once a day

async function loadKnowledge() {
  if (_kb && Date.now() - _at < TTL) return _kb; // serve from cache
  _kb = await fs.readFile(KB_PATH, 'utf-8');
  _at = Date.now();
  return _kb;
}

// inject into every model call:
const knowledge = await loadKnowledge();
const system = `You are a support assistant. Answer ONLY from the
knowledge below. If the answer isn't here, say you don't know.

--- KNOWLEDGE BASE ---
${knowledge}
--- END ---`;

const response = await ai.models.generateContent({
  model: MODEL,
  config: { temperature: 0.2, systemInstruction: system },
  contents: [{ role: 'user', parts: [{ text: userQuery }] }],
});
```

The one rule that prevents hallucination. Always tell the model, in caps, to answer **only** from the document and to **admit when it doesn't know**. Without this, the model fills gaps with confident invention. The file is the model's *only* permitted source of facts — say so explicitly, every call.

Where this plugs into the pipeline

The Markdown file *replaces* the fetch-data steps. Your pipeline gets shorter:

```
USER MESSAGE
|
[0] CACHE CHECK
|
```

```
[1] (optional) ANALYZE — only if you route across several .md topics
|
[2] ANSWER — Gemma reads the injected knowledge.md and replies
    (text answer, or designed cards if you still render UI)
▼
[9] STREAM & CACHE
```

Steps [3]–[8] (fetch, shape, aggregate, hydrate) simply don't exist — there are no rows to shape. If your assistant is purely conversational (text in, text out), you can use the simple `/api/chat` style endpoint that calls Gemma with a system prompt and message history, no pipeline at all.

Practical tips

- **Mind the size.** The whole file is sent on every call, so every word costs tokens. Keep it tight — a few pages, not a novel. If it grows past ~10–15 pages, it's time to split it (see below) or move to a database/search index.
- **Split by topic if it's big.** Keep one small `index.md` listing topics, then load only the relevant topic file after a cheap routing call. This is "poor man's retrieval" and keeps prompts small.
- **Version it in git.** The `.md` file is just text — commit it, review changes in pull requests, and you get a full history of what the assistant "knew" and when.
- **Editable by non-engineers.** A product owner can update the knowledge by editing Markdown — no code deploy if you read it fresh (or on a short TTL). This is a real advantage over hardcoded answers.
- **Cache, but allow refresh.** Read once and cache (like above). For frequently-edited files, drop the TTL to minutes, or expose an admin endpoint that busts the cache on demand.

Hybrid is fine — and common. You can do both at once: a Markdown file for stable context (definitions, business rules, tone) *and* a database for live numbers. Our system already runs this way — hand-written catalog Markdown for grounding plus BigQuery for the actual figures. Start with just the `.md` file; add a data source later only if and when you need live data.

12. Clarification & follow-up flows

PLAIN

When the question is vague

A good assistant asks before guessing — but only when needed, and never twice for the same thing. The model returns `action: "clarify"` with a question and 3–4 option chips. The user taps one; that answer is appended to a running `clarificationHistory`, and the pipeline re-runs with the extra context. Because every option is drawn from your real catalog, the user can never pick a dead end.

```
vague query → Gemma: "clarify" → UI shows question + option chips
→ user taps "Sales" → append {q,a} to history → re-run pipeline
→ now enough context → "route" → real answer
```

When they want to change the answer on screen

This is the feature static chats simply cannot do. With an answer already rendered, the follow-up goes through the **fused** call (Part 8.3) which picks one of four paths:

User says...	Action	What happens
"show only the KPIs"	<code>edit_structural</code>	Model returns the surviving cards; we re-attach their original data. No database hit.
"top 5 territories"	<code>edit_data_change</code>	Model supplies a SQL tweak; pipeline re-queries from step [3].
"summarize this in bullets"	<code>qa_answer</code>	Model writes a text reply using the on-screen data; no new dashboard.
"now show me network outages"	<code>new_report</code>	Treated as a fresh question; full pipeline runs.

The clever bit: data survival on edits. When the model rewrites the card layout for a structural edit, it never sees the heavy data arrays (we strip them to save tokens). So after the edit we rebuild a lookup keyed by `renderType + title` and re-attach the original rows. The user reorders charts; the data rides along untouched.

13. Caching, retries & safety nets BUILD

An LLM system has more failure modes than a static one. Three layers keep it dependable.

CACHING — DON'T PAY TWICE

Key the cache on a stable hash of *everything that affects the answer*: the query text, the persona, the clarification history, and any prior context. On a hit, skip steps [3]–[8] and replay the stored components. TTL of ~5 minutes is plenty for interactive use; the data catalog gets a longer TTL (10 min) and a 24h hard refresh.

RETRIES — SURVIVE A FLAKY MODEL

Every model call is wrapped in `withRetry` (Part 7): on a 429 (rate limit) or 500 (server error), wait and retry up to 3 times with exponential backoff. Other errors throw immediately — no point retrying a malformed request.

FALLBACKS — NEVER SHOW A DEAD END

Failure	Safety net
Model call fails after retries	Deterministic fallback: build the next clarifying question from known context using plain code, no model.
Model returns empty/garbage cards	Fallback card generator produces a basic table from the data.
Model invents a table/option	Code validates against the real catalog and drops invalids.
Query returns 0 rows	Recovery path suggests alternative tables/domains.
Model lowercases a column name	<code>fixColumnCasing</code> maps it back to the real schema.
Chart type unsuitable for data	Validator falls back to a Table view with a notice.

Rule: every model call needs a non-model answer behind it. The model is the preferred path, never the only path. If it's down, the user should still get *something* useful. This is the difference between a demo and a product.

14. Security & configuration MUST

- **Keys live server-side only.** `GOOGLE_AI_API_KEY` and DB credentials go in the backend `.env` / host config — never in frontend code, never in the git repo. If a key ever reaches the browser bundle, it is compromised.
- **Never deploy a `.env` file to cloud hosts.** Set each variable in the platform's config panel (App Service settings, etc.).
- **Lock CORS in production.** Default `*` is fine for local dev; in production set `FRONTEND_ORIGIN` to your exact frontend URL so only it can call the backend.
- **Gate access if needed.** Our backend checks a server-side `ACCESS_PASSWORD` via `/api/auth/verify`; the password never ships to the client.
- **Validate model output before using it.** Treat everything the model returns as untrusted input: parse defensively, validate against your catalog/schema, never `eval` it, never run model-supplied SQL without sanitising the override.
- **Least-privilege data access.** The service account should be read-only (Data Viewer + Job User), scoped to the one dataset the app needs.

The model-supplied SQL caveat. The `edit_data_change` path lets the model return a SQL fragment (e.g. `ORDER BY REVENUE DESC LIMIT 5`). Keep these to constrained, append-only clauses, validate them, and run them under a read-only account. Never let a raw model string become an unguarded full query.

15. Testing & evaluation BUILD

Static chats are tested by clicking every branch. LLM chats need a different approach because the output varies. Build an **evaluation set**: a list of example questions paired with what should happen.

```
testCases = [
  { query: 'show revenue trend',      expect: { route: 'fact_sug_monthly_rollup',
    chartContains: 'LineChart' } },
  { query: 'hello',                  expect: { action: 'clarify' } },
  { query: 'compare T-007 and T-001', expect: { chartContains: 'ComparisonCard',
    mentionsEntities: ['T-007', 'T-001'] } },
];
```

Run the set after every prompt change and score the hit rate. This turns prompt tuning from guesswork into measurement — you can prove a change helped instead of hoping. Our repo ships exactly this as a runnable benchmark.

WHAT TO ASSERT (NOT EXACT STRINGS — BEHAVIOURS)

- Did it route to the correct table?
- Did it clarify when (and only when) it should?
- Did the chosen chart type match the intent?
- Did it use the specific entity's numbers, not the global average?
- Is the JSON valid and renderable?

Manual smoke tests matter too. Keep a short list of "trap" queries — vague, multi-part, off-topic, typo-ridden — and run them by hand before each release. The eval set catches regressions; trap queries catch the weird stuff users actually type.

16. Cost & performance PLAIN

Two things drive cost: **how many model calls** you make and **how big each prompt** is. Our design attacks both.

Lever	Effect
Deterministic-First (3 model calls, not 9)	Fewer calls per question
Fused follow-up (1 call does classify + edit)	Halves follow-up cost
Fast path (skip model when query is already explicit)	Some questions cost \$0 in model calls
Send 10–20 sample rows, not the whole table	Smaller, cheaper prompts
Strip data arrays before edit calls	Edit prompts stay tiny
5-minute cache	Repeat questions are free + instant
Gemma instead of a frontier model	Lower per-token price for routine decisions

Latency. Streaming (SSE) doesn't make the work faster, but it makes it *feel* 3–4× faster because the user sees progress within ~1 second instead of staring at a blank screen for the full duration.

17. Migration checklist

BUILD

The recommended order. Each phase is shippable on its own.

PHASE 1 — FOUNDATIONS

- ☐ Get a `GOOGLE_AI_API_KEY` from Google AI Studio.
- ☐ Stand up a Node + Express backend with one endpoint and a `.env`.
- ☐ Make one successful Gemma call in JSON mode; add `stripThinkTags` + `extractJSON` + `withRetry`.

PHASE 2 — GROUND IT IN YOUR DATA

- ☐ Write your `DATA_SOURCES` map (domain → report → table/source → KPIs).
- ☐ Write a fetch function that returns `rows[]` for a given source.
- ☐ Add the startup availability probe.

PHASE 3 — THE BRAIN (3 MODEL CALLS)

- ☐ Implement `analyzeQuery` (route vs. clarify) with the closed-list prompt + code validation.
- ☐ Implement `generateReport` (design the cards) with the EXACT_COLUMNS rule.
- ☐ Implement the fused `classifyAndEditReport` for follow-ups.

PHASE 4 — THE PLUMBING (PLAIN CODE)

- ☐ Shape analysis, pre-aggregation, column-casing fix, hydration.
- ☐ Assemble them into the ordered pipeline.

PHASE 5 — THE FRONTEND SWAP

- ☐ Build the `UITreeRenderer` (start with 5 component types).
- ☐ Point the chat input at the backend; delete the static decision tree.
- ☐ Render `clarification` options + `followUp` chips from model output.

PHASE 6 — MAKE IT FEEL ALIVE

- ☐ Add SSE streaming; render components as they arrive.
- ☐ Add the cache.

PHASE 7 — HARDEN

- ☐ Wire every fallback (deterministic clarify, fallback cards, 0-row recovery).
- ☐ Build the eval set; run it; tune prompts.
- ☐ Lock CORS, move secrets to host config, set read-only DB access.

18. Pitfalls & FAQ

PLAIN

COMMON PITFALLS

Pitfall	Fix
Letting the model do everything	Deterministic-First — model only for judgement calls.
Trusting model output blindly	Validate against your real catalog/schema in code, every time.
Sending whole tables to the model	Sample 10–20 rows; attach full data after.
Open-ended prompts	Give closed lists + exact JSON shapes.
No fallback path	Every model call gets a non-model backup.

Column-name mismatches → blank charts	EXACT_COLUMNS rule + casing fix.
Re-asking answered questions	Carry clarification history; tell the model not to repeat.
Keys in frontend code	Server-side only, always.

FAQ

- Do we have to use BigQuery?** No. Any data source works — swap the fetch function. The model never touches the database directly.
- Do we have to use Gemma?** No. The model id is one constant. Gemma is our cost/quality default; swap to Gemini or another model per call if a task needs it.
- Can we keep our existing UI?** Yes — that's the point. You keep the look; you swap the engine and add one renderer.
- How long does a conversion take?** A focused engineer can reach a working Phase-3 prototype in days. Polish (streaming, eval, hardening) is where the rest of the time goes.
- What if the model gives a wrong answer?** It can. That's why answers are grounded in your real fetched data, output is validated, and fallbacks exist. Never let the model state facts it wasn't given data for.
- Is it more expensive than static?** Per message, yes — there's a model call. But Deterministic-First, caching, sampling, and Gemma keep it modest, and the capability gain is enormous.

19. Glossary — plain English

PLAIN

Term	Means
LLM	Large Language Model — software that reads and writes human language and makes judgement calls from it.
Gemma	Google's open model family. Our brain. Accessed via the Google AI API.
Prompt	The instruction sheet you hand the model. The system prompt is the rules; the user content is the question + data.
System instruction	The standing rules given to the model on every call (tone, allowed options, output format).
JSON mode	A setting that forces the model to reply in machine-readable structured data instead of prose.
Token	A chunk of text (~¾ of a word). Models are billed per token, in and out — smaller prompts cost less.
Temperature	A 0–1 dial for randomness. Low = consistent/predictable; higher = more creative/varied.
Intent	What the user actually wants (a trend, a comparison, a single number, an explanation).
Routing	Deciding which data source answers the question.
Clarification	The model asking a follow-up question when the request is too vague to answer well.
Pipeline	The ordered chain of steps a message passes through on the backend.
Deterministic	Plain code that always does the same thing — the opposite of the model's variability.
Hydration	Attaching the real data rows to the chart shells the model designed.
SSE	Server-Sent Events — streaming the answer to the browser piece by piece.
Grounding	Forcing the model to work only from your real data and a closed list of options, so it can't make things up.
Render type	The name of a UI widget (LineChart, KPICard...) the model is allowed to choose.
Fallback	The non-model backup that runs when a model call fails or returns junk.
CORS	The rule controlling which website is allowed to call your backend.

Static Chat → LLM Chat Conversion Guidebook • Reference implementation: Conversational UI (Generative BI Report Hub) • Stack: React + Node/Express + Google Gemma ([gemma-4-31b-it](#)) + BigQuery • For deeper code detail see [CHAT_FLOW_ARCHITECTURE.md](#) and [backend/src/services/llmHandler.ts](#) in the repo.